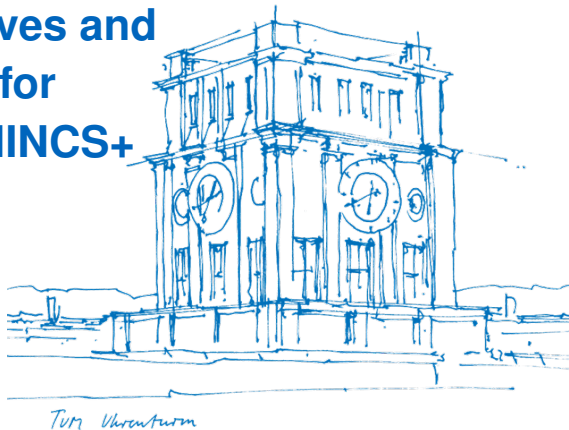


The Impact of Hash Primitives and Communication Overhead for Hardware-Accelerated SPHINCS+

Patrick Karl, Jonas Schupp, Georg Sigl

Technical University of Munich
TUM School of Computation, Information and Technology

April 10, 2024



- 1 Architectures for Hardware Acceleration
- 2 Benchmarking Architectures
- 3 Case-Study: SPHINCS+

1 Architectures for Hardware Acceleration

2 Benchmarking Architectures

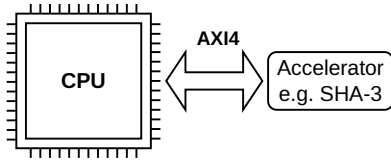
3 Case-Study: SPHINCS+

Architectures for Hardware Accelerators

- Post-Quantum Cryptography (PQC) uses computationally intensive subroutines
 - Huge percentage for hashing and PRNG (mostly SHA-3)
 - Other examples NTT-based polynomial multiplication (mostly for lattice schemes)
- HW acceleration obviously makes sense
- ✗ BUT accelerators often under-utilized!

Architectures for Hardware Accelerators

- “Traditional way”: loosely-coupled
 - Connect via system bus
 - Common also for e.g., FPGA SoCs

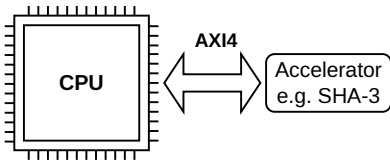


- ✓ Suitable for *heavy* computations
- ✗ Can suffer from bus-latency

Architectures for Hardware Accelerators

■ “Traditional way”: loosely-coupled

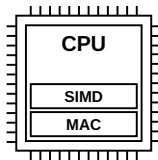
- Connect via system bus
- Common also for e.g., FPGA SoCs



- ✓ Suitable for *heavy* computations
- ✗ Can suffer from bus-latency

■ “Integrated way”: tightly-coupled

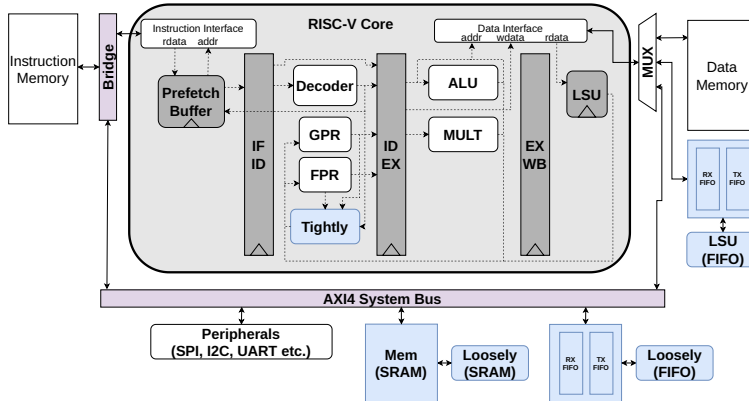
- Additional ISA extensions
- Requires open-source access



- ✓ Resource efficient
- ✓ No extra cost of data transfer
- ✗ Toolchain adaptations/custom assembly

Architectures for Hardware Accelerators

- A lot of possibilities with open-source RISC-V implementations!



Architectures for Hardware Accelerators

■ Efficiency of architectures depends on

1. Computational complexity of operations
2. Amount of data to be transferred

■ Contribution

- ☐ Benchmarking different architectures on PULPino¹ (32-bit RISC-V microcontroller)
- ☐ Case-Study for SPHINCS+ [Ber+19]
- ☐ Alternative functions (TurboShake, Ascon-Xof(a))

¹ <https://github.com/pulp-platform/pulpino>

Architectures for Hardware Accelerators

■ Efficiency of architectures depends on

1. Computational complexity of operations
2. Amount of data to be transferred

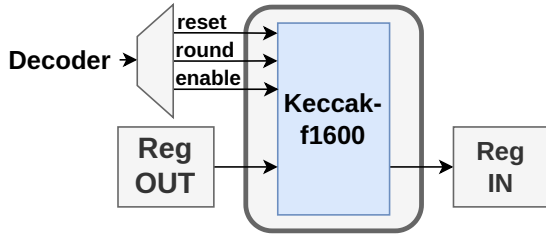
■ Contribution

- ☐ Benchmarking different architectures on PULPino¹ (32-bit RISC-V microcontroller)
- ☐ Case-Study for SPHINCS+ [Ber+19]
- ☐ Alternative functions (TurboShake, Ascon-Xof(a))

■ **NOT** a full SPHINCS+ co-processor!

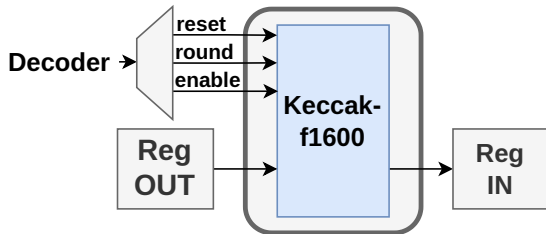
¹ <https://github.com/pulp-platform/pulpino>

Architectures for Hardware Accelerators

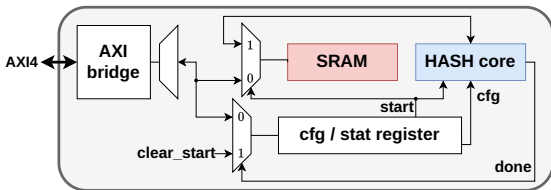


(a) Tightly-coupled as in [FSS20] or [SP21]

Architectures for Hardware Accelerators



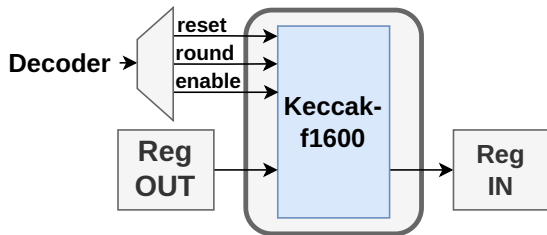
(a) Tightly-coupled as in [FSS20] or [SP21]



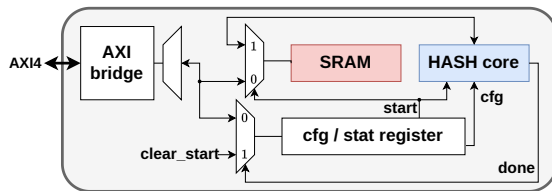
(b) Loosely-coupled with SRAM

- SRAM write → Compute → SRAM read
 - Huge latency as write/compute is not parallelized
 - Replace SRAM with FIFOs

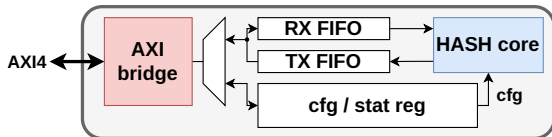
Architectures for Hardware Accelerators



(a) Tightly-coupled as in [FSS20] or [SP21]

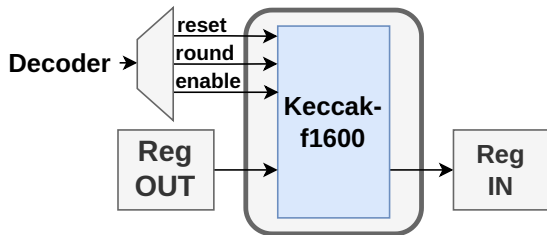


(b) Loosely-coupled with SRAM

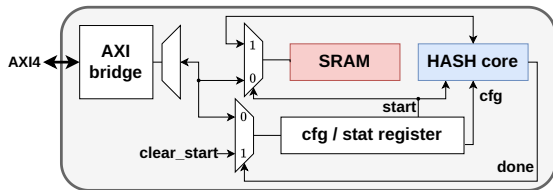


(c) Loosely-coupled with FIFOs

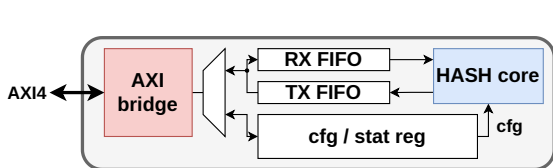
Architectures for Hardware Accelerators



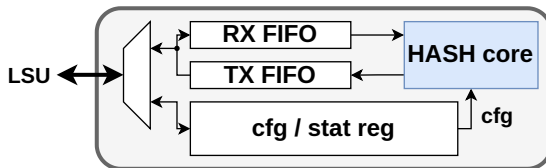
(a) Tightly-coupled as in [FSS20] or [SP21]



(b) Loosely-coupled with SRAM



(c) Loosely-coupled with FIFOs



(d) LSU-coupled with FIFOs

1 Architectures for Hardware Acceleration

2 Benchmarking Architectures

3 Case-Study: SPHINCS+

Benchmarking Setup

We investigated two artificial scenarios:

■ Hash scenario

- ☐ Short outputs, e.g. 32B
- ☐ Sweep over different input lengths

→ Classical file hashing, digesting etc.

■ CSPRNG scenario

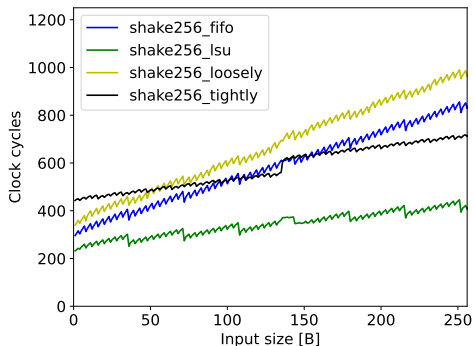
- ☐ Short seed input, e.g. 16B
- ☐ Sweep over different output lengths

→ Widely present in PQC schemes for sampling, masking etc.

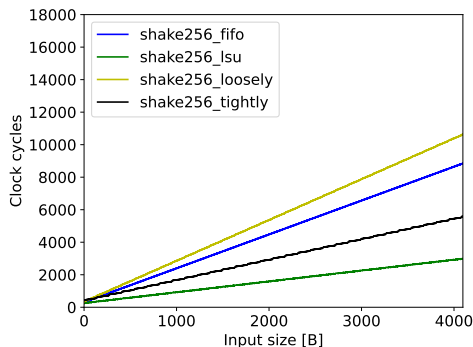
■ Measured on Xilinx UltraScale+ (ZCU102)

- ☐ Running on 150MHz
- ☐ Critical path not affected by accelerators

Results Hash Scenario



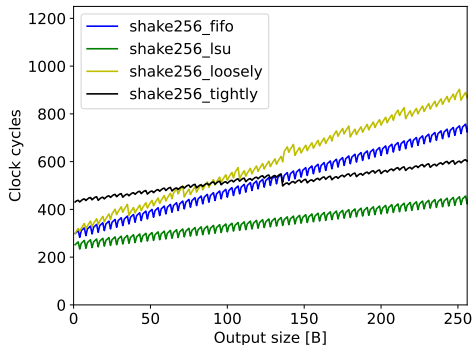
(a) Hash scenario short



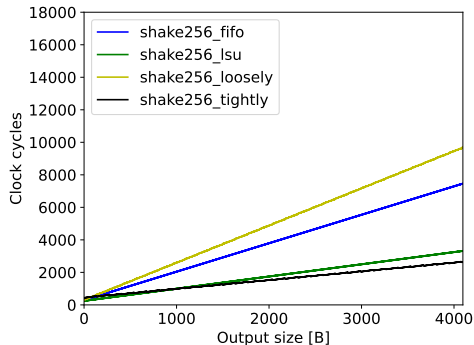
(b) Hash scenario long

- Tightly-coupled efficient for small inputs, suffers from “software absorption”
- LSU-coupled version efficient due to parallel absorption/data transfer

Results CSPRNG Scenario



(a) CSPRNG scenario short



(b) CSPRNG scenario long

■ LSU-coupled version still efficient in general

Takeaway's

- **Tightly-coupled** accelerator suffers from absorption phase
 - inefficient as GPR and FPR is required
 - absorbing into FPR requires data movement to GPR
 - ✓ very efficient in squeezing phase
 - better in **CSPRNG** scenario for large output data

Takeaway's

■ **Tightly-coupled** accelerator suffers from absorption phase

- ☐ inefficient as GPR and FPR is required
- ☐ absorbing into FPR requires data movement to GPR
- ✓ very efficient in squeezing phase
- better in **CSPRNG** scenario for large output data

■ **LSU-coupled** accelerator efficient in absorption phase

- ☐ absorption and data transfer to accelerator in parallel
- ☐ less efficient in squeezing phase (data must be read from accelerator)
- ✓ still overall good performance
- better in **Hash** scenario

Takeaway's

■ **Tightly-coupled** accelerator suffers from absorption phase

- inefficient as GPR and FPR is required
- absorbing into FPR requires data movement to GPR
- ✓ very efficient in squeezing phase
- better in **CSPRNG** scenario for large output data

■ **LSU-coupled** accelerator efficient in absorption phase

- absorption and data transfer to accelerator in parallel
- less efficient in squeezing phase (data must be read from accelerator)
- ✓ still overall good performance
- better in **Hash** scenario

■ **Loosely-coupled** (FIFO and SRAM) obviously slower due to additional bus latency

1 Architectures for Hardware Acceleration

2 Benchmarking Architectures

3 Case-Study: SPHINCS+

Case-Study: SPHINCS+

- Software reference (NIST round 3 submission) C code
 - Replaced calls to shake256 with drivers for accelerators
 - Cycles averaged over 50 runs
 - Here focus on NIST security level 1 (for other parameter sets see paper)

Case-Study: SPHINCS+

■ Software reference (NIST round 3 submission) C code

- ☐ Replaced calls to shake256 with drivers for accelerators
- ☐ Cycles averaged over 50 runs
- ☐ Here focus on NIST security level 1 (for other parameter sets see paper)

Architecture	Keygen	Sign	Verify
shake256_sw	145,119	3,602,835	201,209
shake256_lsu	1,725	42,598	2,458
shake256_tightly	2,902	71,921	4,091

Table 1 Kilo cycles for SPHINCS+ 128f-simple variant.

Architecture	Keygen	Sign	Verify
shake256_sw	9,288,740	70,607,335	69,400
shake256_lsu	109,770	837,742	852
shake256_tightly	184,914	1,410,992	1,419

Table 2 Kilo cycles for SPHINCS+ 128s-simple variant.

Case-Study: SPHINCS+

■ Software reference (NIST round 3 submission) C code

- ☐ Replaced calls to shake256 with drivers for accelerators
- ☐ Cycles averaged over 50 runs
- ☐ Here focus on NIST security level 1 (for other parameter sets see paper)

Architecture	Keygen	Sign	Verify
shake256_sw	145,119	3,602,835	201,209
shake256_lsu	1,725	42,598	2,458
shake256_tightly	2,902	71,921	4,091

Table 1 Kilo cycles for SPHINCS+ 128f-simple variant.

Architecture	Keygen	Sign	Verify
shake256_sw	9,288,740	70,607,335	69,400
shake256_lsu	109,770	837,742	852
shake256_tightly	184,914	1,410,992	1,419

Table 2 Kilo cycles for SPHINCS+ 128s-simple variant.

■ LSU-coupled best performance

- ☐ More than $\times 84$ compared to software for signing
- ☐ $\approx 40\%$ faster than tightly-coupled architecture

Alternative Primitives

- What about a “small” version of SPHINCS+ for security level 1?
 - LWC winner Ascon [Dob+21] would be suitable (Ascon-Xof)
 - Also sponge construction, only 12 rounds but smaller rate (64 bit)

Alternative Primitives

- What about a “small” version of SPHINCS+ for security level 1?
 - LWC winner Ascon [Dob+21] would be suitable (Ascon-Xof)
 - Also sponge construction, only 12 rounds but smaller rate (64 bit)

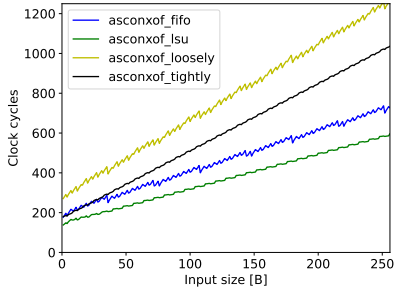
- What about round-reduced versions (TurboShake [Ber+23], Ascon-Xofa)?
 - TurboShake 12 instead of 24 rounds
 - Ascon-Xofa 8 instead of 12 rounds for “inner” permutations

Alternative Primitives

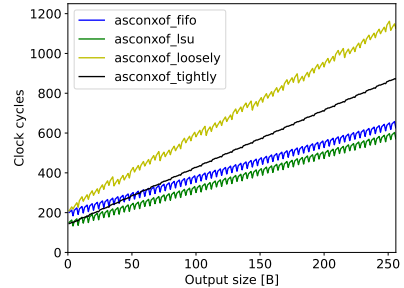
- What about a “small” version of SPHINCS+ for security level 1?
 - LWC winner Ascon [Dob+21] would be suitable (Ascon-Xof)
 - Also sponge construction, only 12 rounds but smaller rate (64 bit)

 - What about round-reduced versions (TurboShake [Ber+23], Ascon-Xofa)?
 - TurboShake 12 instead of 24 rounds
 - Ascon-Xofa 8 instead of 12 rounds for “inner” permutations
- Same benchmarks and evaluation for SPHINCS+

Ascon-Xof Benchmark

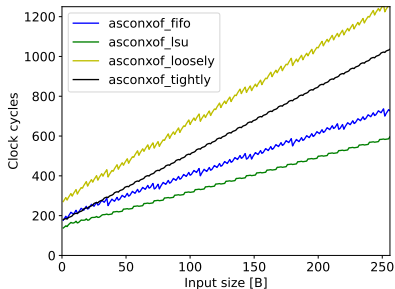


(a) Hash scenario

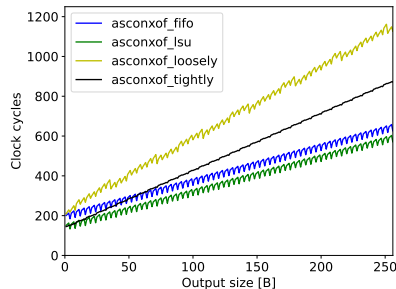


(b) CSPRNG scenario

Ascon-Xof Benchmark



(a) Hash scenario

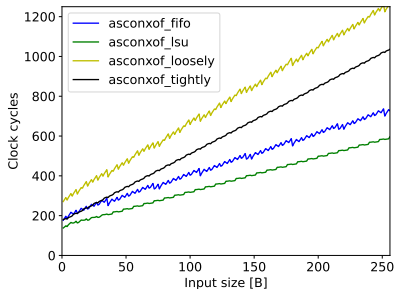


(b) CSPRNG scenario

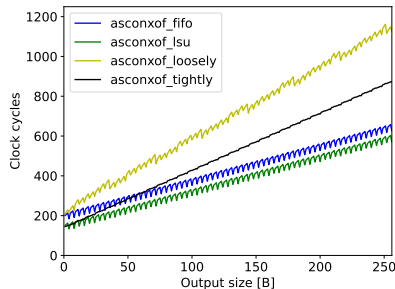
■ asconxof_fifo faster than asconxof_tightly

□ Data transfer not an issue

Ascon-Xof Benchmark



(a) Hash scenario



(b) CSPRNG scenario

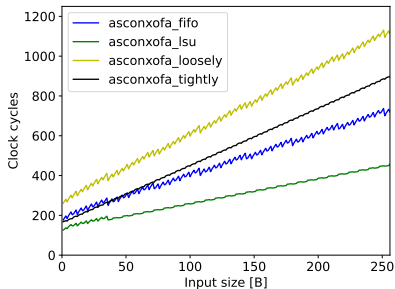
■ asconxof_fifo faster than asconxof_tightly

□ Data transfer not an issue

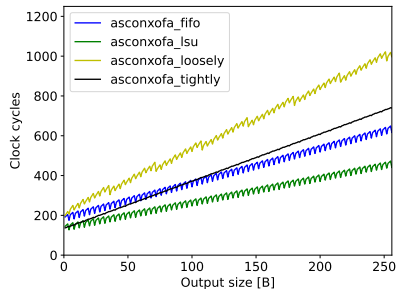
■ asconxof_fifo almost as fast as asconxof_lsu

□ Permutation is bottleneck

Round-Reduction: Ascon-Xofa Benchmark



(a) Hash scenario



(b) CSPRNG scenario

- Round-reduction improves on LSU-coupled version
 - Now different slope visible (compared to asconxofa_fifo)
 - Effect of bus-latency becomes visible

Architecture	Keygen	Sign	Verify
shake256_sw	145,119	3,602,835	201,209
shake256_lsu	1,725	42,598	2,458
shake256_tightly	2,902	71,921	4,091
asconxof_sw	112,543	2,793,036	159,128
asconxof_lsu	1,733	42,868	2,475
asconxof_tightly	2,298	56,896	3,282

Table 3 Kilo cycles for SPHINCS+ 128f-simple variant.

Architecture	Keygen	Sign	Verify
shake256_sw	9,288,740	70,607,335	69,400
shake256_lsu	109,770	837,742	852
shake256_tightly	184,914	1,410,992	1,419
asconxof_sw	7,203,739	54,787,744	55,470
asconxof_lsu	110,308	843,462	864
asconxof_tightly	146,232	1,116,889	1,147

Table 4 Kilo cycles for SPHINCS+ 128s-simple variant.

■ Using Ascon-Xof instead of shake256:

- ✓ More than 22 % faster in SW
- ✓ Tightly-coupled version about 20 % faster

■ Comparing LSU-coupled versions

- ☐ Ascon-Xof slightly slower
- ☐ Permutation is bottleneck (bandwidth limit)

Alternative Primitives: Application to SPHINCS+ II

Architecture	Keygen	Sign	Verify
shake256_sw	145,119	3,602,835	201,209
shake256_lsu	1,725	42,598	2,458
shake256_tightly	2,902	71,921	4,091
asconxof_sw	112,543	2,793,036	159,128
asconxof_lsu	1,733	42,868	2,475
asconxof_tightly	2,298	56,896	3,282

Table 5 Kilo cycles for SPHINCS+ 128f-simple variant.

Architecture	Keygen	Sign	Verify
t_shake256_sw	85,584	2,124,535	118,879
t_shake256_lsu	1,684	41,594	2,401
t_shake256_tightly	2,848	70,574	4,014
asconxofa_sw	82,716	2,052,962	116,680
asconxofa_lsu	1,571	38,851	2,242
asconxofa_tightly	2,136	52,884	3,049

Table 6 Kilo cycles for SPHINCS+ 128f-simple variant.

■ Round-reduction directly visible in SW

- ✓ About 41 % speed-up for turbo-shake256
- ✓ About 26 % speed-up for Ascon-Xofa

■ Limited gain for accelerators

- 2 % for turbo-shake256
- ✓ About 10 % for Ascon-Xof
- ✓ Ascon-Xofa fastest overall result!

Takeaway's

- Interfacing and data transfer cost has a huge impact on overall performance
 - Under-utilization degrades performance

Takeaway's

- Interfacing and data transfer cost has a huge impact on overall performance
 - Under-utilization degrades performance
- Tightly-coupled architecture can resolve that
 - But is inherently slower for some computations (still SW involved)

Takeaway's

- Interfacing and data transfer cost has a huge impact on overall performance
 - Under-utilization degrades performance
- Tightly-coupled architecture can resolve that
 - But is inherently slower for some computations (still SW involved)
- Round-reduction has a limited benefit for HW acceleration if data transfer dominates
- Ascon-Xof(a) is an attractive alternative for SPHINCS+ at NIST security level 1!

References I

- [Ber+19] D. J. Bernstein et al. “The SPHINCS+ Signature Framework”. In: *Proceedings of the 2019 ACM SIGSAC Conference on Computer and Communications Security*. ACM, 2019.
- [FSS20] T. Fritzmann, G. Sigl, and J. Sepúlveda. “RISQ-V: Tightly Coupled RISC-V Accelerators for Post-Quantum Cryptography”. en. In: *IACR Transactions on Cryptographic Hardware and Embedded Systems Volume 2020* (2020), Issue 4.
- [SP21] S. Steinegger and R. Primas. “A Fast and Compact RISC-V Accelerator for Ascon and Friends”. In: *Smart Card Research and Advanced Applications*. Ed. by P.-Y. Liardet and N. Mentens. Cham: Springer International Publishing, 2021, pp. 53–67.
- [Dob+21] C. Dobraunig et al. “Ascon v1.2: Lightweight Authenticated Encryption and Hashing”. In: *J. Cryptol.* 34.3 (2021), p. 33.
- [Ber+23] G. Bertoni et al. *TurboSHAKE*. Cryptology ePrint Archive, Paper 2023/342. Version 20230308:172924. <https://eprint.iacr.org/2023/342>. Mar. 2023.